Universidad Diego Portales

Laboratorio 2: Pilas y Filas

Problema: Web Crawler

Estructuras de Datos y Algoritmos
2026-1

Profesores:
Matías Aguilera
Marcos Fantoval
Cristián Llull
Karol Suchan

Trabajo en equipos de 3 estudiantes

1. Introducción

En este laboratorio usted y su equipo darán el salto hacia los Tipos de Datos Abstractos (TDA) y el diseño de *software* Orientado a Objetos. El objetivo es experimentar cómo las Interfaces nos permiten encapsular el comportamiento de un TDA separándolo de su estructura interna.

1.1. Motivación

La *World Wide Web* se organiza en archivos que contienen hipervínculos a otros archivos conectados a través de internet. De aquí se desprende que *navegar por internet* consiste en explorar estos hipervínculos. Un *web crawler* es un programa que explora sistemáticamente la *World Wide Web* de forma automática. Se utiliza tanto para descargar, analizar como para indexar contenido. Los buscadores, como Google, lo utilizan para este último propósito, actualizando sus índices según ciertos criterios.

Dentro de los tipos de archivos que se pueden encontrar en internet, podemos encontrar los siguientes estándares:

- HTML: "HyperText Markup Language", es decir, un lenguaje que utiliza etiquetas para estructurar el contenido. Es el código estándar para desplegar y estructurar contenido en páginas web.
- XML: "eXtensible Markup Language". Al igual que en HTML, utiliza etiquetas para jerarquizar el contenido. Fue concebido para, además de mostrar contenido, gestionar datos y permitir la interacción con máquinas.
- XHTML: se desarrolló para hacer que HTML fuera extensible y flexible, lo que permite trabajar con otros formatos, como XML. Además, es un lenguaje más estricto, lo que facilita el análisis de la estructura de un documento (p. ej., una página web) y su validación.

En este laboratorio se le pedirá que implemente un simulador de *web crawler* que verifique el contenido de páginas en formato XHTML.

1.2. Objetivos

Se le pedirá que construya un simulador de *web crawler*, un software que recorre páginas web, extrae enlaces y valida la estructura correcta del código XHTML.

Este laboratorio busca poner en práctica:

- El uso e implementación de las estructuras de datos Pila (Stack) y Fila (Queue).
- La comprensión del concepto de interfaz en Java mediante adaptadores.
- El análisis empírico y teórico (Big-O) de algoritmos.

2. Problema a estudiar

Un *web crawler* navega por internet utilizando una estrategia de "Búsqueda en anchura" (BFS). Para esto, requiere una Fila (*Queue*): toma una URL, lee el archivo al que conduce y enfila todos los enlaces "href" que contenga para explorarlos más tarde.

Simultáneamente, para cada página visitada, el sistema debe verificar que su código XHTML esté correctamente formado. Esto requiere una Pila (*Stack*): al leer el código de izquierda a derecha, cada vez que se encuentre una etiqueta de apertura (p. ej.,), debe hacer push(). Al encontrar una etiqueta de cierre (p. ej.,), debe hacer pop() y verificar su correspondencia.

Usted cuenta con las implementaciones de *Stack* y *Queue* del texto guía (`https://algs4.cs.princeton.edu/code/`). Sin embargo, para entender el impacto del diseño interno de un TDA, usted deberá crear también sus implementaciones propias:

- Una Pila construida internamente con solo un único objeto de tipo Queue.

- Una Fila construida internamente con solo dos objetos de tipo Stack.

El desafío consiste en comprobar cómo estas implementaciones afectan el rendimiento del validador a medida que aumenta el número N de elementos en una página web.

3. Etapas del Laboratorio

3.1. Etapa 1: Interfaces y Adaptadores (Programación)

Deberá programar las interfaces genéricas Pila<T> y Fila<T>, y luego programar cuatro clases que implementen dichas interfaces.

3.1.1. Interfaces

`Pila<T>`

- Métodos:
  - `void push(T ítem)`: Agrega un elemento en la cima de la pila.
  - `T pop()`: Elimina el elemento de la cima de la pila y lo devuelve.
  - `T peek()`: Devuelve, pero no elimina, el elemento de la cima de la pila.
  - `boolean isEmpty()`: Devuelve si la pila está vacía.
  - `int size()`: Devuelve el número de elementos en la pila.

`Fila<T>`

- Métodos:
  - `void enqueue(T ítem)`: Agrega un elemento al final de la fila.
  - `T dequeue()`: Elimina el elemento del inicio de la fila y lo devuelve.
  - `T peek()`: Devuelve, pero no elimina, el elemento del inicio de la fila.
  - `boolean isEmpty()`: Devuelve si la fila está vacía.
  - `int size()`: Devuelve el número de elementos en la fila.

Luego, deberá programar las siguientes 4 clases implementando dichas interfaces:

- PilaPrinceton y FilaPrinceton, simplemente usando herencia a partir de las clases respectivas de *Princeton*[1][2].
- PilaConFila<T>: Debe asegurar el comportamiento LIFO, pero el único atributo permitido en la clase es un objeto de tipo Queue<T>.
- FilaConPilas<T>: Debe asegurar el comportamiento FIFO utilizando únicamente dos objetos de tipo Stack<T>.

[1] `https://algs4.cs.princeton.edu/code/edu/princeton/cs/algs4/Queue.java.html`
[2] `https://algs4.cs.princeton.edu/code/edu/princeton/cs/algs4/Stack.java.html`

3.1.2. Web Crawler

En esta etapa deberá implementar una clase `WebCrawler`. Debe inspirarse en la clase WebCrawler de Princeton[3]. Su clase debe tener atributos `Pila<String> pila` y `Fila<String> fila`, y los siguientes métodos:

- `WebCrawler(Pila<String> pila, Fila<String> fila)`: Recibe la pila y la fila a ser usadas internamente.
- `boolean esXHTMLValido(String xhtml)`: Recibe un documento XHTML en formato de String. Debe utilizar la `pila` para verificar si el XHTML es válido y devolver dicha información. Se puede limitar a revisar las siguientes reglas[4]:
  - Incluir `<!DOCTYPE>` es mandatorio.
  - El atributo xmlns en `<html>` es obligatorio.
  - Cada etiqueta de apertura de un elemento debe estar acompañada por una etiqueta de cierre (por ejemplo " " con " "). Los elementos deben cerrarse en el orden LIFO, en el mismo orden en que se abrieron.
  - Los elementos deben escribirse en minúsculas.
- `boolean chequearURL(String url)`: Recibe un URL, lee el contenido de la página usando la clase In[5] de Princeton, y lo valida usando `esXHTMLValido`. Devuelve el resultado de la validación.
- `boolean chequeaURLs(String urlInicial, int limite)`: Recibe un URL desde el que iniciar el crawl y un límite de número de páginas a revisar. Su método debe:
  - Mantener un registro de los URL ya visitados usando la clase Set[6] de Princeton, para evitar ciclos infinitos (que la página A apunte a la B, y la B a la A).
  - Al revisar cada página, extraer los URL referenciados en elementos `<a>` (a través del atributo `href`) que contiene y agregarlos a la `fila` si no han sido vistos previamente. También debe validar la página usando `chequearURL`.
  - Detener la exploración inmediatamente cuando se alcance el `limite` de páginas validadas, o cuando la fila quede vacía.
  - Devolver la información de si todas las páginas visitadas contienen XHTML válido.
- `static void main(String[] args)`: Crea una `Pila` y una `Fila` y las usa para crear un objeto de la clase `WebCrawler`. Luego, use su método `chequeaURLs` para revisar las 100 primeras páginas alcanzables desde `https://eit.udp.cl/` y las 200 primeras de `https://crawler-test.com`.

3.2. Etapa 2: Análisis Empírico Offline

El análisis empírico debe realizarse eliminando factores externos que nos impidan medir correctamente el tiempo que tarda nuestro algoritmo en realizar *web crawling*. Por lo tanto, se le pide realizar el análisis empírico de forma *offline* utilizando datos sintéticos. Así, por un lado, se puede medir el tiempo dedicado exclusivamente a revisar las páginas web y, por otro, se evita que la red se bloquee al realizar demasiadas peticiones (los firewalls suelen bloquear bots).

Deberá implementar una clase `Experimento.java` que siga el método empírico de *Doubling Ratio*.

Experimento 1: Evaluación de la Pila

- Para cada tamaño $N = 2^i$, con $i \in \{10, 11, 12, 13, 14, 15\}$, genere un *String* que simule un documento XHTML con N etiquetas de apertura anidadas seguidas de N etiquetas de cierre. Por ejemplo, para $N = 4$: `«!DOCTYPE html> <html xmlns=''> <div> <p> <a> </a> </p> </div> </html>"`. Puede usar el siguiente método:

[3] `https://algs4.cs.princeton.edu/42digraph/WebCrawler.java.html`
[4] No son todas. Si quiere ver todas las reglas, visite: `https://www.w3schools.com/htmL/html_xhtml.asp`
[5] `https://introcs.cs.princeton.edu/java/84network/In.java.html`
[6] `https://algs4.cs.princeton.edu/35applications/SET.java.html`

```java
public static String generarXHTMLSintetico(int N) {
    StringBuilder sb = new StringBuilder();
    sb.append("<!DOCTYPE html>\n<html xmlns='http://...'>\n");
    for (int i = 0; i < N-1; i++) sb.append("<div>\n");
    for (int i = 0; i < N-1; i++) sb.append("</div>\n");
    sb.append("</html>");
    return sb.toString();
}
```

- Utilice `StopwatchCPU`[7] para medir el tiempo que toma su método `esXHTMLValido()` al procesar este *String*, primero con su `PilaPrinceton` y luego con su `PilaConFila`.

Experimento 2: Evaluación de la Fila

- Agregue a la clase `WebCrawler` el método `simularCrawlerOffline(int N)` que simula la exploración sin internet.

```java
public static void simularCrawlerOffline(int N, Fila<String> fila) {
    fila.enqueue("url_raiz");
    for (int i = 0; i < N; i++) {
        if (!fila.isEmpty()) fila.dequeue();
        // Simulamos que encontramos 2 links por pagina
        fila.enqueue("link1_" + i);
        fila.enqueue("link2_" + i);
    }
}
```

- Utilice `StopwatchCPU` para medir el tiempo de este proceso utilizando `FilaPrinceton` y luego `FilaConPilas`.

Reporte experimental

1. Utilice la clase `Out`[8] de Princeton para exportar los tiempos de ambos experimentos a un archivo CSV estructurado (ej. `N, PilaPrinceton, PilaDeFila, FilaPrinceton, FilaDePilas`).
2. Ejecute los experimentos 100 veces cada uno y reporte:
   a. Mínimo
   b. Máximo
   c. Promedio
   d. Desviación estándar
   e. Coeficiente de variación (desviación estándar / promedio)

3.3. Etapa 3: Análisis Teórico y Comparación

En un archivo de Excel, tabule los datos del CSV y calcule los estadísticos y el ratio (T(2N)/T(N)). Genere gráficos que comparen el rendimiento de PilaPrinceton versus PilaDeFila y FilaPrinceton versus FilaDePilas.

En su informe final, determine teóricamente el orden de crecimiento Big-O para los métodos `push` y `pop` de sus implementaciones de `Pila` y `enqueue` y `dequeue` de sus implementaciones de `Fila`. Discuta si los resultados empíricos de Excel respaldan su hipótesis teórica.

[7] https://algs4.cs.princeton.edu/code/edu/princeton/cs/algs4/StopwatchCPU.java.html
[8] https://introcs.cs.princeton.edu/java/stdlib/Out.java.html

3.3.1. Reglas para medir bien:

1. No imprimir en la consola dentro de las mediciones.
2. No mezclar la generación de datos y la medición en el mismo bloque.
3. La medición debe incluir solo el tiempo de ejecución del método analizado (no el guardado en archivo ni la creación de estructuras de datos auxiliares).

3.3.2. Análisis en Excel

Debe entregar un archivo `analisis.xlsx`.
Estructura del archivo

1. Una pestaña por cada tamaño (6 en total).
2. Una pestaña final llamada `Resumen`.

Qué debe incluir cada pestaña de tamaño En cada pestaña de tamaño, para cada método, incluya:

1. Los 100 tiempos medidos (microdatos).
2. Estadísticas básicas:
   a. Mínimo
   b. Máximo
   c. Promedio
   d. Desviación estándar
   e. Coeficiente de variación (desviación estándar / promedio)
3. Un histograma de los tiempos.

3.4. Qué debe incluir la pestaña Resumen

1. Una tabla con los **promedios** por tamaño de cada método.
2. Un gráfico **log-log**:
   - Eje x: tamaño n.
   - Eje y: tiempo promedio.
3. Cálculo de ratios entre tamaños consecutivos y sus logaritmos base 2.
4. Interpretación breve: ¿a qué valor parece converger el logaritmo?

4. Informe

Además del código y el Excel, deben entregar un **informe en PDF** que documente todo el proyecto.

4.1. Estructura obligatoria

El informe debe incluir, en este orden:

1. **Introducción**
   - Descripción general del desafío.
   - Objetivos del laboratorio.
   - Qué entregables se construyeron (visión general).
2. **Etapa 1: Implementación de filas, pilas y web crawler**
   - Qué clases tiene su programa y para qué sirve cada una.
3. **Etapa 2: Análisis empírico**
   (I) **Estructura del software**
   - Cómo se mide el tiempo y cómo se escriben los archivos CSV.

   (II) **Desafíos enfrentados**
   - Qué fue lo más difícil (por ejemplo: extraer las etiquetas con expresiones regulares, simular el *crawler* offline, aislar el tiempo de medición).
   - Cómo lo resolvieron (criterios, pruebas, decisiones).

   (III) **Análisis empírico**
   - Qué muestran los promedios, histogramas, gráficos log-log.
   - Qué concluyen con la estrategia de duplicación.
4. **Etapa 3: Análisis teórico y Comparación**
   (I) **Excel**
   - Qué contiene cada pestaña (tamaños).
   - Qué contiene el Resumen y cómo leerlo.

   (II) **Análisis**
   - Big-O de cada método y su justificación.

   (III) **Comparación**
   - Tabla o resumen que compare la teoría con el experimento.
   - Discusión argumentada.
5. **Conclusiones**
   - Qué aprendieron.
   - Qué método crece más rápido y por qué.
   - Una reflexión breve sobre por qué sirve comparar teoría y práctica.

5. Entregables

Cada equipo debe entregar (vía Canvas) una carpeta comprimida `.zip` con:

1. Código fuente completo del programa Java del análisis empírico.
2. Archivos CSV generados (uno por tamaño).
3. Archivo `analisis.xlsx`.
4. Informe en PDF.

Importante: si falta algún entregable, el laboratorio se considera incompleto.

Parte	Puntaje
Etapa 1: Programa Java y su correctitud	20
Etapa 1: Uso de clases de *Princeton* (StopwatchCPU, Out, In, Set, Queue, Stack, modelo DoublingRatio)	5
Etapa 2: Excel por tamaño (microdatos + estadísticos + histograma)	5
Etapa 2: Resumen (tabla promedios + log-log + ratios + log2 + interpretación)	5
Etapa 3: Análisis teórico Big-O (justificación clara)	5
Etapa 4: Comparación teoría vs. experimento (discusión y conclusiones)	10
Informe (estructura, claridad, orden, redacción y evidencia)	10
Total	**60**

Cuadro 1: Rúbrica de evaluación del Laboratorio 2.

6. Rúbrica

7. Recomendaciones finales

1. Comience temprano: medir y graficar requieren tiempo.
2. Prefiera métodos más pequeños con funcionalidad precisa a grandes métodos que agrupan mucha funcionalidad.
3. Asegúrese de que los métodos que escriba funcionen correctamente antes de avanzar a lo siguiente.
4. Pruebe primero con tamaños pequeños (por ejemplo, 8, 16) para validar que su programa funciona, antes de medir.
5. Evite imprimir durante la medición.
6. En el informe, privilegie explicaciones simples y concretas: qué hicieron, qué observaron y qué concluyen.